



Tecnológico de Monterrey

Tecnológico de Monterrey CEM

Inteligencia artificial avanzada para la ciencia de datos

Módulo 2

Grupo 601

Momento de Retroalimentación: Módulo 2

Saúl Caballero Castor A01751748

Jorge Adolfo Ramírez Uresti

2 de septiembre del 2026

1. Descripción del algoritmo

Para esta retroalimentación, se implementó una red neuronal feedforward multicapa entrenada mediante backpropagation y gradient descent, sin usar ninguna biblioteca de aprendizaje máquina (solo numpy para operaciones matriciales). El código se encuentra en *neural_network.py*.

El algoritmo incluye, todo implementado manualmente:

- **Forward propagation:** cada etapa calcula:

$$Z = W \cdot A_{prev} + b$$

En donde:

- **Z:** Salida de la neurona (sin función de activación)
- **W:** Matriz de pesos (con tamaño: número de neuronas en la capa x número de neuronas en la capa anterior)
- **A_{prev} :** Salida de la capa anterior (ya con función de activación) (con tamaño: número de neuronas de la capa anterior x número de simples)
- **b:** vector de bias de la capa (con tamaño: número de neuronas de la capa x 1)

Posteriormente se aplica la función de activación antes de pasar a la siguiente capa.

- **Función de pérdida:** *binary cross-entropy*, adecuada para clasificación binaria con salida en probabilidad.
- **Backpropagation:** Cálculo de gradientes (dW , db) capa por capa usando la regla de la cadena, incluyendo la simplificación:

$$dZ = A - Y$$

Para la combinación de sigmoid + cross-entropy en la capa de salida, y:

$$dZ = dA - \text{relu}'(Z)$$

Para las capas ocultas.

- **Actualización de parámetros:** gradient descent:

$$W = W - \text{learning}_{rate} * dW$$

2. Arquitectura de la red

Tabla 1: Arquitectura de capas de NN

Capa	Neuronas	Activación
Entrada	2	-
Oculto	4	ReLU
Salida	1	Sigmoid

Hiperparámetros de entrenamiento:

- **Épocas:** 3000
- **Learning rate:** 0.1

3. Dataset

Se usó el problema XOR, un ejemplo no linealmente separable (no resoluble por un solo perceptrón, por lo que requiere al menos una capa oculta):

Tabla 2: Tabla de verdad XOR.

X1	X2	Y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

Entrenamiento y prueba: dado que el XOR tiene únicamente 4 combinaciones posibles de entrada, se usó el mismo conjunto (las 4 combinaciones) tanto para entrenar como para evaluar el modelo, ya que es el universo completo de casos posibles del problema.

4. Resultados de entrenamiento

Evolución de la pérdida (binary cross-entropy) durante el entrenamiento:

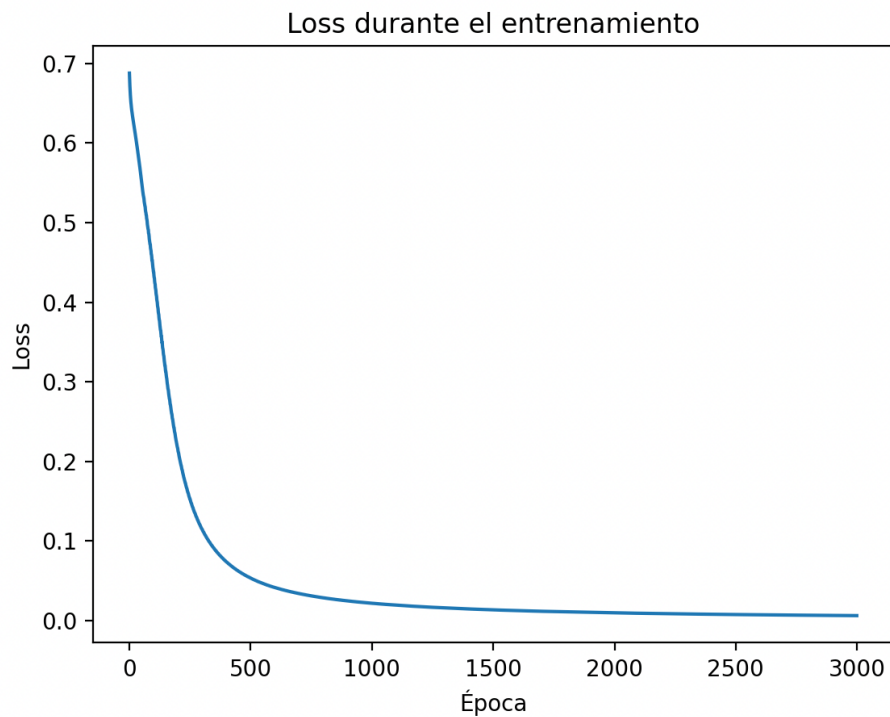


Ilustración 1: Gráfica de loss durante el entrenamiento

Predicciones finales del modelo (probabilidad de salida) contra el valor real:

```
(.venv) scaballero@mac M2 % python3 nerual_network.py
loss: 0.6879946680309668
loss: 0.05295780662630997
loss: 0.02140372636866014
loss: 0.013138637074094314
loss: 0.009363144645205669
loss: 0.007232787287237353
Final loss: 0.005867983959462556
predicciones: [[0.01758617 0.99780101 0.99785173 0.00136701]]
```

Ilustración 2: Salida en terminal loss + predicciones

Tabla 3: Tabla comparativa. Valor real vs Valor predicho

X1	X2	Y real	Predicción (probabilidad)	Predicción (redondeada)
0	0	0	0.0175	0
0	1	1	0.9978	1
1	0	1	0.9978	1
1	1	0	0.0013	0

Las 4 predicciones coinciden exactamente con los valores reales de XOR, con alta confianza (probabilidades muy cercanas a 0 o 1, no ambiguas cerca de 0.5).

5. Matriz de confusión y métricas

```
Matriz de confusión:
[[2 0]
 [0 2]]

Accuracy: 1.0
Precision: 1.0
Recall: 1.0
Specificity: 1.0
F1: 1.0
```

Ilustración 3: Salida en terminal. Matriz de confusión y métricas

	Predicho: 0	Predicho: 1
Real: 0	2 (TN)	0 (FP)
Real: 1	0 (FN)	2 (TP)

Tabla 4: Matriz de confusion

Métrica	Valor
Accuracy	1.0
Precision	1.0
Recall	1.0
Specificity	1.0
F1-score	1.0

Tabla 5: Métricas de desempeño

6. Análisis y conclusión

El modelo alcanzó un desempeño perfecto (todas las métricas en 1.0) sobre las 4 combinaciones del problema XOR, con una pérdida final muy baja (0.0064), lo que indica que la red no solo clasificó correctamente sino que lo hizo con alta confianza en cada predicción.

Este resultado confirma que la arquitectura implementada (una capa oculta con activación no lineal ReLU) es capaz de resolver un problema que no es linealmente separable, algo que un modelo sin capas ocultas (como una regresión logística simple) no podría lograr. Esto valida que el backpropagation y el forward pass fueron implementados correctamente, ya que la red logró ajustar sus pesos para capturar la relación no lineal entre las entradas y la salida esperada.

Al ser un dataset de solo 4 ejemplos, las métricas perfectas no son evidencia de una gran capacidad de generalización, sino de que el modelo memorizó correctamente el único conjunto de casos posibles del problema. Adicionalmente, se observó que la convergencia depende de la inicialización aleatoria de los pesos: en algunas corridas el entrenamiento se estanca en un mínimo local (pérdida ~ 0.35) en lugar de converger al mínimo global.